

Тема 3. Работа с ORM Sequelize и моделями данных

Задание: настроить подключение к SQLite и описать модель пользователя.

Пошаговое руководство:

1. В `db.js` импортировать Sequelize, создать экземпляр с `dialect: sqlite`, `storage:` путь к файлу базы, `logging: console.log`. Экспортировать по умолчанию.
2. В `user.js` импортировать `DataTypes` и `sequelize`. Вызвать `sequelize.define` с именем `users`, описать поля `id`, `login`, `password`, `name`, `tel`, `email`, `role`. Для каждого поля указать тип, `allowNull`, `primaryKey`, `autoIncrement`, `defaultValue`. В третьем аргументе задать `timestamps: false`. Экспортировать модель.
3. В `index.js` импортировать `sequelize` и `Users`, экспортировать их фигурными скобками.

Контрольные вопросы:

1. Что такое ORM и какую проблему разработки решает Sequelize в экосистеме Node.js?
2. Опишите назначение параметров `dialect`, `storage` и `logging` при создании нового экземпляра Sequelize.
3. Какие опции необходимо задать в описании поля `id`, чтобы оно служило первичным ключом и автоматически увеличивалось при добавлении записей?
4. Что делает параметр `timestamps: false` в третьем аргументе метода `define` и в каких случаях его отключение оправдано?
5. Почему в файле `index.js` используется именованный экспорт `{ sequelize, Users }`, а не экспорт по умолчанию?

Тема 4. Сервер, middleware и синхронизация базы

Задание: собрать базовый `server.js`, подключить `middleware` и настроить запуск.

Пошаговое руководство:

1. Импортировать `express`, `cookieParser`, `{ sequelize, Users }` из `models/index.js`.
2. Создать `app = express()`, задать `PORT = 3000`.
3. Подключить `middleware` в строгом порядке: `app.use(express.static('public'))`, `app.use(express.json())`, `app.use(cookieParser())`.
4. Вызвать `sequelize.sync()` для создания таблиц при первом запуске.
5. Написать заглушку для запуска: `await app.listen(PORT, () => console.log(...))`.

Контрольные вопросы:

1. Что такое middleware в Express и почему порядок их подключения через `app.use()` критически важен?
2. Какую задачу выполняет `express.json()` и что произойдёт с `req.body` в контроллере, если переместить этот middleware после объявления маршрутов?
3. Зачем вызывается `sequelize.sync()` перед запуском сервера и в каких ситуациях его использование считается рискованным для продакшен-среды?
4. Как параметр `httpOnly` влияет на безопасность cookie-файлов и от какого типа клиентских атак он защищает?
5. Почему перед `app.listen()` используется ключевое слово `await` и как это связано с настройкой `"type": "module"`?